PeakMeister 2.0

server.R — Task Boundary Map

Source: AmirMazouchi/PeakMeister2.0_UI • branch profiling/initialize-run-timings • commit 709b1bb0997c28519ed8eb144c43ffe62e553f7b.

Open the reviewed server.R on GitHub

Scope: the complete 6,096-line timing-enabled server.R. This document reproduces the task-boundary review; no refactoring was performed.

All start and end lines are inclusive. They refer to this exact commit and differ from the earlier master references because timing instrumentation was added. Some ranges overlap because a larger task contains smaller tasks, or because tasks are grouped by responsibility rather than file order.

These are task boundaries. Some blocks share variables with surrounding code, so turning them into functions will require defining their inputs and outputs.

1. Application setup and About tab

Task / existing function	Start	End
Configure upload-size limit and timeout	15	18
Initialize stored Plotly objects	23	24
Initialize integration boundary positions	26	27
Display disclaimer	31	33
Render and display README	34	36
Display update history	37	39
Display license	40	42

The enclosing server() function runs from lines 21–6095.

2. User-supplied parameters

Task / existing function	Start	End
Initialize metabolite mass-list state	49	62
Initialize reference-mass state	65	77
Initialize processing-parameter state	80	91
Initialize user-supplied migration-index state	95	101
Register initial parameter-table displays	104	107
Read the three worksheets from uploaded parameter Excel file	111	128
Reset parameter datasets when the uploaded file is absent	131	137
Add a metabolite row and reset its input fields	141	191
Delete selected metabolite row	194	199
Clear metabolite list	201	203
Display metabolite table	205	206
Add a reference-mass row and reset its input fields	209	257
Delete selected reference-mass row	260	266
Clear reference-mass list	268	270
Display reference-mass table	272	273
Add a processing-parameter row and reset its input fields	276	304
Delete selected processing-parameter row	307	313
Clear processing parameters	315	317
Display processing-parameter table	319	320
Initialize project-information state	326	329
Store project information entered by the user	331	336
Display stored project information	338	343
Clear project information and its input fields	346	358
Read uploaded migration-index Excel file	363	369
Add a migration-index row and reset its input fields	372	405
Delete selected migration-index row	408	416
Clear migration-index list	419	422
Display migration-index table	425	426
Download current mass lists and parameters as Excel	431	445

3. Engine: file selection and upload

Task / existing function	Start	End
Initialize uploaded-file and pseudo-file state	451	454
Register uploaded mz5 files, display their names, and optionally trigger autorun	456	479
Clear uploaded-file and pseudo-file lists	482	486
Display editable pseudo-file-name table	491	493
Apply edits to pseudo-file names	496	500
Read pseudo-file names from Excel	503	510
Generate empty pseudo-mz5 files and register them	514	533
Add an empty pseudo-file-name row	536	541

4. Initialize Run: preparation and timing

The complete Initialize Run handler spans lines 546–2531. Its per-file loop spans lines 787–2521.

Task / existing function	Start	End
Require an uploaded mz5 file	547	549
Create a uniquely named Results folder	551	557
Define timing recorder, stage transitions, CSV persistence, and total timing	559	604
Initialize run timing and connect stage messages to progress display	605	610
Create Plots and Data directories	612	616
Collect target names, reject duplicates, and determine target/injection counts	618	631
Create target-specific plot directories when using Metabolite format	633	638
Save a copy of the input mass lists and parameters	640	647
Assemble supplied metabolite and reference migration-time tables	651	654
Identify the reference standard preceding each metabolite	656	671
Identify the reference standard following each metabolite	673	688
Calculate migration indexes or relative migration times	690	732
Assemble the migration-index summary table	734	740
Validate and apply user-supplied migration-index overrides	743	766
Export migration-index summary CSV	768	771
Collect uploaded file paths/names and initialize Plotly collection	774	777
Define helper for saving collected Plotly objects	781	783
Start per-file progress reporting and create the file’s Data directory	787	796

5. Initialize Run: raw data to eie_df

Task / existing function	Start	End
Create temporary mz5 copy	799	802
Read mz5 spectra into run_data and unlock its assay-data environment	804	823
Read calibration settings and validate the Yes/No choice	825	848
Estimate a reference mass's measured position and mass error — existing calibration_parameters()	868	909
Iterate through spectra and obtain both reference-mass measurements	913	941
Fit mass-error correction model	943	948
Apply correction to spectrum m/z values	950	958
Calculate target extraction windows from m/z and ppm tolerance	978	986
Inspect spectra, handle invalid m/z values, and rebuild sorted spectra	988	1045
Extract electropherograms using chromatogram()	1047	1054
Assemble time and target-intensity columns into eie_df	1056	1065
Validate smoothing choice and optionally smooth each intensity column	1067	1113

The complete optional calibration block is lines 825–968. The extraction-to-processing handoff is after line 1065, before optional smoothing.

6. Initialize Run: internal-standard peak processing

The complete internal-standard processing stage spans lines 1115–1443.

Task / existing function	Start	End
Detect candidate peaks and collect boundaries, apexes, and intensities	1125	1165
Filter candidates by allowed migration-time range	1167	1171
Integrate candidate peaks and subtract baseline area	1173	1201
Name peak columns and retain candidates for later recovery	1203	1215
Calculate full width at half maximum and filter by width tolerance	1217	1256
Retain peaks with the largest areas up to the injection-count cutoff	1258	1262
Calculate spacing limits and identify abnormal peak gaps	1264	1289
Correct one abnormal gap, recovering a candidate or creating a placeholder	1291	1367
Correct two or three abnormal gaps	1369	1420
Combine processed internal-standard peak tables	1423	1434
Extract standard apex times for metabolite-time prediction	1436	1443

7. Initialize Run: metabolite peak processing

Task / existing function	Start	End
Detect candidate metabolite peaks and collect boundaries/intensities	1455	1501
Filter candidates by minimum peak width	1503	1509
Integrate candidates, subtract baseline, and name peak columns	1511	1547
Predict migration times from indexes and detected reference standards	1549	1576
Assign peaks within migration windows; create placeholders when absent	1578	1621
Create placeholders for predictions beyond the recorded run time	1623	1647
Resolve duplicate peak assignments with progressively narrower windows	1650	1723
Correct assignments using expected spacing between injections	1725	1863
Combine processed metabolite peak tables	1865	1877
Estimate noise thresholds and annotate <LOD and NPD peaks	1879	1932
Detect configured interferences and annotate overlapping peaks	1934	1986
Combine standard/metabolite peaks and their comment tables	1988	2002

Within the noise stage, lines 1906–1908 already define the small noise_calculation() function.

8. Initialize Run: plotting and result export

Task / existing function	Start	End
Initialize per-target plot-data collection	2010	2013
Build peak annotations	2019	2024
Build integration shading data and assign peak numbers	2026	2053
Add baseline values to integration shading	2055	2070
Assemble per-target traces, annotations, labels, and axis limits	2072	2084
Define static electropherogram plot — plot_function()	2088	2123
Define interactive electropherogram plot — plot_function_plotly()	2126	2182
Create plot directories for Sample format	2184	2207
Save internal-standard PNGs in Sample format	2212	2233
Save metabolite/reference composite PNGs for relative-time targets	2235	2276
Save composite PNGs for migration-index targets	2278	2308
Save PNGs organized by metabolite	2310	2336
Refresh final target’s stored plot-data entry	2338	2347
Build and collect internal-standard Plotly objects	2350	2368
Build and collect metabolite Plotly objects	2370	2418
Publish collected plots to reactive state	2420	2426
Build peak-area report, apply annotations, and accumulate files	2429	2449
Build migration-time report and accumulate files	2452	2463
Save per-file plot_list_data.RData	2465	2467
Open/initialize SQLite, replace existing file records, and write results	2469	2496
Remove temporary mz5 copy	2499	2504
Export accumulated peak-area and migration-time CSVs	2506	2514
Save accumulated Plotly objects and finish file-stage timing	2516	2519
Finalize run timing or record failure/interruption	2523	2530

9. Visualization: state and database helpers

These tasks already contain several named functions.

Task / existing function	Start	End
Initialize current results, working metadata, and undo state	2538	2568
Resolve selected Results directory — main_folder	2571	2574

Task / existing function	Start	End
Load selected file's .RData results — plot_list_data	2577	2590
Query available target/plot names — filtered_plot_names	2594	2608
Track modified plots and render invalidation	2611	2615
Construct SQLite path — get_db_path()	2621	2623
Check database existence — db_exists()	2626	2628
Open database with write-ahead logging — open_db()	2631	2636
Create database tables and indexes — initialize_db()	2639	2722
Safely obtain a column, supplying missing values — safe_col()	2725	2727
Write one file's results transactionally — write_file_to_db()	2730	2871
Read one target's trace from .RData and editable data from SQLite — read_plot_data_from_db()	2874	2941
Record an edit audit entry — log_edit()	2944	2950
Migrate existing .RData results into SQLite — migrate_rdata_to_db()	2953	3013
Reconstruct wide result tables from SQLite — load_wide_data_from_db()	3017	3108
Assemble a file's complete plot-data collection — load_plot_list_from_db()	3111	3121

9a. Database writer: individual storage tasks

These are subtasks within write_file_to_db(), listed in section 9.

Task / existing function	Start	End
Transform and store annotations	2736	2754
Transform and store integration shading/baselines	2756	2774
Store axes and plot labels	2776	2790
Convert peak measurements from wide to long format and store them	2792	2809
Convert peak areas to long format and store them	2811	2828
Convert migration times to long format and store them	2830	2843
Serialize and store migration indexes	2845	2864
Commit or roll back the write transaction	2866	2870

10. Visualization: selecting results and displaying plots

Task / existing function	Start	End
Populate source-file selector from uploaded files	3127	3129
Discover Results directories — list_results_folders()	3133	3137
Poll for new Results directories	3140	3147
Update Results-directory selector	3150	3155
Initialize database connection state	3158	3158
Switch Results directory: close old database, open selected database, or offer migration	3161	3203
Handle migration confirmation, progress, outcome, and connection	3206	3256
Close database when the Shiny session ends	3259	3264
Load selected file’s database results into reactive state	3267	3321
Load results when a database connection becomes available	3324	3379
Display available plot names	3386	3388
Load and render selected electropherogram, preserving zoom	3392	3430
Initialize and switch between zoom and rectangle-drawing modes	3436	3455
Initialize zoom state	3458	3461
Capture zoom changes and reset on autorange	3464	3479
Reset zoom when selecting a different target	3482	3487
Capture dragged integration lines or rectangle-selected boundaries	3490	3548
Display integration boundary positions	3552	3557
Query and display selected target’s peak annotations	3561	3583

11. Visualization: editing and saving peaks

Task / existing function	Start	End
Load .RData results for editing — loaded_plot_list_data	3590	3608
Persist a target's edits to SQLite and .RData, then refresh state — save_plot_data()	3611	3716
Reload result tables from SQLite — reload_plot_data()	3720	3736
Initialize editing state from .RData when switching files	3741	3760
Mark selected peaks as not detected and remove their integration shading	3763	3847
Mark all interior peaks as not detected, retaining first and last	3851	3930
Mark every peak as not detected	3933	4008
Restore previous metadata and save it to .RData	4012	4056
Show peak-number input dialog — peak_input_modal()	4566	4575
Integrate between selected boundaries and update measurements — manual_peak_integration()	4579	4708
Handle manual integration using table selection or open the dialog	4712	4734
Handle manually entered peak number and integrate	4737	4757
Initialize baseline-adjustment modes	4761	4762
Show peak-number dialog for baseline adjustment	4765	4772
Recalculate one peak using a clicked baseline — adjust_individual_baseline()	4775	4882
Activate individual-baseline adjustment	4886	4891
Capture baseline click and apply/request peak selection	4894	4918
Handle manually entered peak number for baseline adjustment	4921	4924
Show whole-plot baseline dialog — show_adjust_all_baselines_modal()	4929	4939
Recalculate all peaks using a common baseline — adjust_plot_baseline()	4942	5018
Activate whole-plot baseline adjustment	5022	5028
Render electropherogram inside baseline dialog	5031	5058
Capture clicked baseline and apply it to all peaks	5061	5078
Close whole-plot baseline dialog	5081	5083
Display three target-selection tables for EIE comparison	5087	5097
Load and display two or three stacked EIE plots	5101	5150
Mark selected peaks as interfered and save the changes	5153	5232

11a. Edit saving: individual persistence tasks

These are subtasks within save_plot_data(), listed in section 11.

Task / existing function	Start	End
Replace target annotations in SQLite	3617	3632
Replace target integration data in SQLite	3634	3652
Replace target peak measurements in SQLite	3654	3674
Replace target peak areas in SQLite	3676	3687
Append edit-log entry	3689	3690
Save .RData backup	3692	3703
Reload target data and trigger display refresh	3705	3715

12. Visualization: rebuilding plots and reports

Task / existing function	Start	End
Define static electropherogram plot for regeneration — plot_function()	4062	4099
Define interactive electropherogram plot for visualization — plot_function_plotly()	4102	4158
Rebuild peak-area CSV from SQLite — regenerate_metabolite_peak_areas()	4161	4187
Rebuild and compress all saved Plotly objects — regenerate_plots()	4190	4244
Regenerate changed PNGs in Sample or Metabolite layout — regenerate_ggplot()	4247	4403
Rebuild migration-time CSV from annotations — regenerate_metabolite_migration_times()	4406	4432
Define loading-screen message collection	4437	4468
Display additional-options dialog	4471	4486
Coordinate changed-plot regeneration, loading screen, and peak-area export	4489	4535
Display list of modified plots	4538	4540
Handle standalone peak-area CSV rebuild	4543	4551
Handle standalone migration-time CSV rebuild	4554	4562

regenerate_plots() is defined, but the current regeneration button calls the peak-area and PNG regeneration functions; it does not call this Plotly rebuild helper.

13. Downstream processing: data loading and transformation

Task / existing function	Start	End
Read peak-area and migration-time CSVs — grab_metabolite_files()	5237	5252
Initialize downstream datasets	5255	5256
Load selected Results folder’s CSVs into state	5259	5266
Resolve downstream Results folder	5269	5272
Populate downstream Results/file selectors	5275	5278
Parse m/z and name from target identifiers — extract_mass()	5283	5287
Assemble m/z-versus-migration-time data for selected file/injection	5290	5323
Display interactive m/z-versus-migration-time plot	5326	5338
Display injection/peak-number selector	5341	5344
Show normalization-reference selection dialog	5349	5359
Validate, normalize, update state, and save normalized areas	5362	5405
Perform row-wise normalization — nested normalize_peak_areas()	5377	5394
Replace missing values with minimum/5 or zero, then save	5409	5443
Read metadata, match file/injection to sample IDs, merge, and save	5449	5509
Extract one file’s block of peak-area rows — nested filter_data_by_file_name()	5487	5491

14. Downstream processing: quality control and statistics

Task / existing function	Start	End
Populate control-chart reference selector	5514	5519
Generate raw-area control chart with mean and ±3 SD limits	5522	5550
Generate processed-area control chart with mean and ±3 SD limits	5554	5575
Calculate per-column coefficient of variation — calculate_cv()	5582	5585
Define reactive CV calculation for current area data — cv_data	5588	5593
Perform Pareto scaling — pareto_scale()	5597	5600
Remove zero-variance columns — remove_zero_variance()	5603	5608
Validate data, scale, and calculate PCA — perform_pca()	5611	5636
Define reactive PCA calculation for current data — pca_data	5639	5645
Parse QC categories/exclusions, split samples, filter invalid columns, calculate CVs and PCA	5649	5714
Calculate and display technical/biological CV summary table	5721	5738
Prepare and display CV-versus-m/z plot	5740	5765
Build, save, and display QC/non-QC PCA plot	5767	5808
Display current downstream peak-area table	5814	5824

The compute_cv button handler enclosing the CV table and both plots spans lines 5718–5809.

15. Downstream processing: batch correction

Task / existing function	Start	End
Install/load optional batch-correction packages	5828	5849
Read uploaded batch-order CSV	5852	5855
Obtain area data and batch metadata; log-transform metabolite columns	5861	5872
Configure correction strategies and reference-sample groups	5874	5889
Apply batch correction to each metabolite	5891	5901
Name correction results and reverse log transformation	5904	5910
Export transformed and original-scale corrected results	5912	5914

The complete batch-correction button handler spans lines 5859–5916.

16. Reporting: MetaboAnalyst export

Task / existing function	Start	End
Initialize reporting-matrix state	5920	5920
Validate input data and identify output folder	5925	5930
Remove file/peak identifiers, transpose data, and obtain sample IDs	5932	5936
Parse user-defined sample categories	5938	5943
Assign a class to each sample	5945	5963
Insert and order sample-ID and CLASS rows	5965	5970
Store and export MetaboAnalyst matrix	5972	5979
Display generated matrix	5984	5990

The complete matrix-generation handler spans lines 5923–5980.

17. Drug Analysis: data import

Task / existing function	Start	End
Initialize patient, DrugBank, and TWOSIDES state	5999	6001
Read uploaded adherence-results Excel file	6005	6010
Load TWOSIDES database	6015	6017
Load cached DrugBank data or offer XML parsing	6019	6040
Show loading screen while parsing DrugBank	6042	6069
Select DrugBank fields and parse XML database	6071	6083
Cache parsed DrugBank data, update state, and report completion	6085	6092

This final section currently imports data; it does not yet calculate drug adherence.

Structural observation

Calculations, file/database access, reactive-state updates, and UI actions are mixed within the same blocks. Several calculations already have functions, but those functions still read Shiny state directly. That dependency needs to be captured when choosing the first task to extract.